JUnit Testing Fundamentals

1. What is JUnit, and why is it important for unit testing?

A: JUnit is a simple framework in Java used for writing and running **unit tests**. It is important because it lets developers easily check if small, individual parts of their code (like one method or one class) work correctly in isolation. This helps catch bugs early and ensures that code changes don't break old features.

2. What is the difference between @Before and @BeforeClass?

A: @BeforeEach (JUnit 5) or @Before (JUnit 4) runs its method **before every single test method** in the class. **@BeforeAll** (JUnit 5) or @BeforeClass (JUnit 4) runs its method **only once** before any of the tests in the entire class start. You use @BeforeAll for expensive setup (like creating a database connection) that can be shared.

3. What is the purpose of the @Test annotation?

A: The purpose of the **@Test** annotation is to mark a Java method as a method that JUnit should execute as a test case. When you run your tests, JUnit looks for all methods marked with @Test and runs them one by one.

4. What is the difference between assertEquals, assertTrue, and assertSame?

A:

- **assertEquals**: Checks if two values are **equal** in content (e.g., the number 5 equals the number 5).
- **assertTrue**: Checks if a condition or expression is **true**.
- **assertSame**: Checks if two object variables point to the **exact same object** in memory (they are identical references).

5. How do you test expected exceptions in JUnit?

A: In modern JUnit 5, you test expected exceptions using the **assertThrows** method. You wrap the code that is expected to throw the error inside a lambda expression. assertThrows verifies that the specified type of exception is indeed thrown when the code runs.

6. What are parameterized tests in JUnit?

A: Parameterized tests allow you to run the **same test method multiple times** using different sets of input data. Instead of writing the same logic in five separate test methods, you provide a list of inputs and expected outputs, and JUnit runs the test with each set automatically.

7. What is a test suite in JUnit?

A: A **test suite** is a way to group and run multiple test classes together at the same time. You create a suite class that lists all the individual test classes you want to execute, allowing you to manage and execute large collections of tests easily.

8. How do you handle timeouts in JUnit?

A: You handle timeouts by specifying a maximum duration for a test method. In JUnit 5, you add the **@Timeout** annotation to the test method, specifying the time (e.g., Timeout(value = 5, unit = TimeUnit.SECONDS)). If the test runs longer than the defined time, JUnit automatically fails the test.

9. How do you structure a test case using AAA (Arrange–Act–Assert)?

A: AAA provides a simple, clear structure for tests:

1. **Arrange:** Set up all the required objects, variables, and preconditions (the setup phase).
2. **Act:** Execute the specific piece of code or method you are testing (the execution phase).
3. **Assert:** Verify that the result or state of the system is what you expected (the verification phase).

Mockito Testing Fundamentals

1. What is Mockito, and why is it used?

A: Mockito is a popular framework used in Java for creating **mock objects** (fake versions of real things) during testing. It is used because it lets you test one small piece of your code (like a Service class) completely **in isolation**, without needing the real database, network, or external services to be running.

2. How do you mock an object in Mockito?

A: You mock an object by using the **Mockito.mock(Class.class)** static method, or by using the annotation **@Mock** on the object's field. Mocking tells Mockito to create a simple dummy version of that class which you can then control during your test.

3. What is the purpose of @Mock and @InjectMocks?

A: The purpose of **@Mock** is to create the fake, controlled objects (dependencies) that your main class needs. The purpose of **@InjectMocks** is to create the actual instance of the class you want to test. Mockito then tries to automatically insert all the objects marked with @Mock into the fields of the @InjectMocks object.

4. How do when() and thenReturn() work?

A: when() is used to tell a mock object exactly what to do when a specific method is called. **thenReturn()** defines the exact fixed result that the mock should give back. For example, you might say: when(repository.findById(1)).thenReturn(userJohn).

5. How do you verify method calls using verify()?

A: You use **verify()** to check if a specific method on a mock object was called, and how many times it was called, during the execution of your test. This ensures that the code under test correctly performed the actions you expected it to.

6. How do you mock exceptions?

A: You mock exceptions by using the **when().thenThrow()** syntax. This tells the mock object to throw a specific error instead of returning a normal value when a certain method is called. This lets you test how your code handles error situations.

7. What is the difference between mock() and spy()?

A: A **mock()** object is a complete fake; it has no real logic or code unless you explicitly tell it what to do. A **spy()** object is a real object that keeps all its original code and behavior, but you can choose to override specific methods if you need to control their output.

8. What is ArgumentCaptor, and how do you use it?

A: ArgumentCaptor is a utility used when you need to check the **exact value** of an argument that was passed to a mock method call. You use it in conjunction with **verify()** to capture the input parameter and then run assertions against that captured value.

TestNG Fundamentals

1. What is TestNG vs JUnit?

A: JUnit is the older, simpler testing framework mainly used for basic **unit tests** (testing single classes). **TestNG** (Test Next Generation) is a more powerful, modern framework designed for complex **integration tests** and end-to-end testing. TestNG offers more advanced features like dependency injection, parallel testing, and grouping tests.

2. Key annotations in TestNG.

A: Key annotations define the setup and teardown logic:

- **@Test**: Marks a method as an executable test case.

- **@BeforeSuite / @AfterSuite**: Runs once before/after all tests in a suite.
- **@BeforeClass / @AfterClass**: Runs once before/after all methods in a test class.
- **@BeforeMethod / @AfterMethod**: Runs before/after every single @Test method.

3. Difference between @BeforeMethod and @BeforeClass.

A: @BeforeClass runs its method only once, before the first test method in that test class starts. **@BeforeMethod** runs its method multiple times—once before *each* and *every* test method within that class. You use @BeforeClass for expensive one-time setup (like starting a server).

4. How to configure test suites using testng.xml.

A: The **testng.xml** file is the main configuration file where you define which test classes or groups to run. It allows you to create **test suites** and set parameters, control parallel execution, and manage listener classes from one central location.

5. What are groups in TestNG?

A: Groups are tags or labels you assign to specific test methods (e.g., @Test(groups = "smoke")). You use groups to run a filtered subset of tests based on those tags (e.g., running only "smoke" tests or only "regression" tests) via the testng.xml file.

6. What is @DataProvider?

A: @DataProvider is used to supply external data to a @Test method. It defines a method that returns a matrix (rows and columns) of test data. The corresponding @Test method will run once for every row of data returned by the provider.

7. What is @Parameters?

A: @Parameters is used to read data from the external **testng.xml** file and inject those values directly into a test method or a @BeforeMethod setup method. This allows you to change settings (like a browser name or URL) without recompiling your Java code.

8. How to skip a test intentionally?

A: You can skip a test intentionally by using the **enabled = false** attribute within the @Test annotation (e.g., @Test(enabled = false)). The test will be recognized by the framework but will be skipped during execution.

9. Soft vs hard assertions.

A: A **hard assertion** immediately stops the test execution if the assertion fails. A **soft assertion** continues executing the test code even if an assertion fails, logging the failure instead. The

test only fails completely at the very end when assertAll() is called. You use soft assertions when you want to execute all checks in a test, even after finding the first bug.

10. How to disable a test?

A: You disable a test by setting the **enabled** attribute to **false** inside the @Test annotation, as mentioned previously (e.g., @Test(enabled = false)).

11. How to prioritize test methods?

A: You prioritize test methods by using the **priority** attribute within the @Test annotation (e.g., @Test(priority = 1)). Tests with a lower priority number run before tests with a higher number. This is useful for running setup or dependency tests first.

RestAssured Fundamentals

1. What is RestAssured and why is it used?

A: RestAssured is a very popular Java library used for testing and validating REST APIs. It is used because it provides a simple, **Domain-Specific Language (DSL)** that makes writing complex API tests feel easy and natural, making testing fast and reliable.

2. How do you set baseURI?

A: You set the base URI by using the static property **RestAssured.baseURI** once at the start of your tests (e.g., RestAssured.baseURI = "http://api.example.com"). This saves you from typing the full server address for every single test call you make.

3. What is given–when–then?

A: given-when-then is the standard structure (DSL) RestAssured uses to make tests readable:

- **given()**: Sets up the request (like headers, body, or parameters).
- **when()**: Sends the actual request (like get("/users") or post("/items")).
- **then()**: Verifies the response (like checking the status code or JSON content).

4. How do you validate status codes?

A: You validate status codes in the **then()** part of the test. You use **.statusCode(200)** to assert that the response status code matches the expected number (e.g., 200 for success or 404 for not found).

5. How do you validate JSON fields?

A: You validate JSON fields using the **jsonPath()** method within the **then()** section. You specify the path to the field you want to check (e.g., body("data.name", equalTo("John"))). This verifies that the field has the correct value.

6. How do you send query/path params?

A: You send **query parameters** (for filtering) using the **.queryParam("key", "value")** method within the **given()** section. You send **path parameters** (for resources) using the **.pathParam("id", "101")** method, and the parameter name is placed inside the URL.

7. How do you send headers?

A: You send headers using the **.header("Header-Name", "value")** method within the **given()** section. This is typically used to send authentication tokens (like Authorization Bearer) or to set the content type.

8. How to send POST with JSON body?

A: You send a POST request with a JSON body by using the **.body(jsonPayload)** method in the **given()** section, and then calling **.post("/resource")** in the when() section. You should also set the Content-Type header to application/json.

9. How to extract values from JSON?

A: You extract values from JSON after the when() call using the **.extract().path("field.path")** method. This pulls a specific value (like a new user ID) from the response body so you can use it in later tests.

10. How to log request/response?

A: You log the request details by adding **.log().all()** immediately after the **given()** keyword. You log the response details by adding **.log().all()** immediately after the **then()** keyword. This prints the full request and response to the console for debugging.